Aim: Defining instance variables using a constructor.

Program:

```
class Student:
   def __init__(self, name, age):
      self.name = name      # instance variable
      self.age = age        # instance variable


# Creating objects
s1 = Student("Rahul", 20)
s2 = Student("Priya", 22)


# Accessing instance variables
print(s1.name, s1.age)
print(s2.name, s2.age)
```

output:

```
Rahul 20
Priya 22


=== Code Execution Successful ===
```

Aim: Write a Python function to calculate the factorial of a number (a non-negative integer). The function accepts in python

Program:

```
def factorial(n):
    if n < 0:
        return "Factorial not defined for negative numbers"
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Example usage
num = 5
print("Factorial of", num, "is", factorial(num))
```

output:

```
Factorial of 5 is 120

=== Code Execution Successful ===
```

Aim: Implement a class Student with information as rollno , class ,name. The information must be entered by the user.

Program:

```
class Student:
    def __init__(self, rollno, student_class, name):
        self.rollno = rollno
        self.student_class = student_class
        self.name = name

    def display(self):
        print("Roll No:", self.rollno)
        print("Class:", self.student_class)
        print("Name:", self.name)



# Taking input from user
rollno = input("Enter Roll Number: ")
student_class = input("Enter Class: ")
name = input("Enter Name: ")

# Creating object
s = Student(rollno, student_class, name)

# Displaying details
s.display()
```

Output:

```
Enter Roll Number: 21

Enter Class: MCA

Enter Name: Bibhu

Roll No: 21

Class: MCA

Name: Bibhu


=== Code Execution Successful ===
```

Aim: Write a python program to create a tuple and to find greatest and smallest element form the tuple.

Program:

```
# Creating a tuple
t = (10, 25, 5, 40, 15)

# Finding greatest and smallest elements
greatest = max(t)
smallest = min(t)

# Displaying results
print("Tuple:", t)
print("Greatest element:", greatest)
print("Smallest element:", smallest)
```

Output:

```
Tuple: (10, 25, 5, 40, 15)
Greatest element: 40
Smallest element: 5

=== Code Execution Successful ===
```

Aim: Write a Python program to ADD, SUBSTRACT, MULTIPLY, AND DIVIDE two Pandas Series.

Program:

```
import pandas as pd


# Creating two Pandas Series
s1 = pd.Series([10, 20, 30, 40])
s2 = pd.Series([5, 10, 15, 20])


# Operations
addition = s1 + s2
subtraction = s1 - s2
multiplication = s1 * s2
division = s1 / s2


# Display results
print("Series 1:\n", s1)
print("Series 2:\n", s2)


print("\nAddition:\n", addition)
print("\nSubtraction:\n", subtraction)
print("\nMultiplication:\n", multiplication)
print("\nDivision:\n", division)
```

Output:

```
Series 1:
0  10
1  20
2  30
3  40
dtype: int64
Series 2:
0   5
1  10
2  15
3  20
dtype: int64

Addition:
0  15
1  30
2  45
3  60
dtype: int64

Subtraction:
0   5
1  10
2  15
3  20
dtype: int64

Multiplication:
0    50
1   200
2   450
3   800
dtype: int64

Division:
0  2.0
1  2.0
2  2.0
3  2.0
dtype: float64

=== Code Execution Successful ===
```

Aim: Write a python program to read an entire text file.

Program:

```
# Opening and reading the entire file
with open("sample.txt", "r") as file:
    content = file.read()

# Displaying file content
print(content)
```

Output:

```
This is a Sample file.
```

Aim: **Write a program to perform different Arithmetic Operations on numbers in Python.**

Program:

```
# Taking input from user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Performing operations
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2

# Handling division safely
if num2 != 0:
    division = num1 / num2
else:
    division = "Undefined (cannot divide by zero)"

# Displaying results
print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
```

Output:

```
Enter first number: 17
Enter second number: 10
Addition: 27.0
Subtraction: 7.0
Multiplication: 170.0
Division: 1.7

=== Code Execution Successful ===
```

Aim: **Write a python program to define a module to find Fibonacci Numbers and import the module to another program.**

Program:

Create a module (fibonacci_module.py)

```
def fibonacci(n): a, b = 0, 1 series = []

for i in range(n):
        series.append(a)
        a, b = b, a + b

    return series
```

Create another program to import and use the module

```
import fibonacci_module

n = int(input("Enter number of terms: "))

result = fibonacci_module.fibonacci(n)

print("Fibonacci Series:", result)
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python main.py
Enter number of terms: 10
Fibonacci Series: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Aim: Write a program in python and create a pandas dataframe and locate the row.

Program:

```
import pandas as pd

# Creating a DataFrame

data = {

   'Name': ['Rahul', 'Priya', 'Amit', 'Neha'],

   'Age': [20, 22, 21, 23],

   'City': ['Delhi', 'Mumbai', 'Pune', 'Chennai']

}

df = pd.DataFrame(data)

# Display DataFrame

print("DataFrame:\n", df)

# Locating a row using index (e.g., index 2)

row = df.loc[2]

print("\nLocated Row (index 2):\n", row)
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python dataframe.py
DataFrame:
     Name  Age     City
0  Rahul   20    Delhi
1  Priya   22   Mumbai
2   Amit   21     Pune
3   Neha   23  Chennai

Located Row (index 2):
 Name    Amit
Age       21
City    Pune
Name: 2, dtype: object
```

Aim: Write a program in python to display a digital clock.

Program:

```
import time


while True:
    current_time = time.strftime("%H:%M:%S")  # format: HH:MM:SS
    print("\rCurrent Time:", current_time, end="")
    time.sleep(1)
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python time.py
Current Time:^C 16:19:04Traceback (most recent call last):
  File "/home/savvy/Program/python/time.py", line 6, in <module>
    time.sleep(1)
    ~~~~~~~~~~^^^
KeyboardInterrupt
```

Aim: Write a program in python to sort the given list without using in-built function.

Program:

Taking input list

lst = [5, 2, 9, 1, 7]

Sorting using Bubble Sort

n = len(lst)

for i in range(n): for j in range(0, n - i - 1): if lst[j] > lst[j + 1]: # swapping lst[j], lst[j + 1] = lst[j + 1], lst[j]

Display sorted list

print("Sorted List:", lst)

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python bubble_sort.py
Sorted List: [1, 2, 5, 7, 9]
```

Aim: Write a program in python to check whether the number entered is Armstrong or not.

Program:

```
# Taking input
        num = int(input("Enter a number: "))

        # Store original number
        temp = num
        order = len(str(num)) # number of digits
        sum = 0

        # Calculate sum of digits raised to power of order
        while temp > 0:
         digit = temp % 10
         sum += digit ** order
         temp //= 10

        # Check Armstrong condition
        if sum == num:
         print(num, "is an Armstrong number")
        else:
         print(num, "is not an Armstrong number")
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python store_number.py
Enter a number: 10
10 is not an Armstrong number

┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python store_number.py
Enter a number: 153
153 is an Armstrong number
```

Aim: Write a Python Program to merge to python dictionaries.

Program:

```
# Creating two dictionaries

dict1 = {'a': 1, 'b': 2}

dict2 = {'c': 3, 'd': 4}


# Merging dictionaries

merged_dict = {}


for key in dict1:

merged_dict[key] = dict1[key]


for key in dict2:

merged_dict[key] = dict2[key]


# Display result

print("Merged Dictionary:", merged_dict)
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python merge.py
Merged Dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Aim: Write a Python Program to multiply two matrices.

Program:

Matrix A

A = [[1, 2, 3], [4, 5, 6]]

Matrix B

B = [[7, 8], [9, 10], [11, 12]]

Result matrix (initialized with zeros)

result = [[0, 0], [0, 0]]

Matrix multiplication

for i in range(len(A)): # rows of A for j in range(len(B[0])): # columns of B for k in range(len(B)): # columns of A / rows of B result[i][j] += A[i][k] * B[k][j]

Display result

print("Resultant Matrix:") for row in result: print(row)

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python matrix.py
Resultant Matrix:
[58, 64]
[139, 154]
```

Aim: Write a Program to perform Regression

Program:

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data (X = independent variable, y = dependent variable)
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict values
y_pred = model.predict(X)
# Display results
print("Predicted values:", y_pred)
print("Slope (Coefficient):", model.coef_)
print("Intercept:", model.intercept_)
```

Output:

```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python scikit-learn.py
Predicted values: [2.8 3.4 4.  4.6 5.2]
Slope (Coefficient): [0.6]
Intercept: 2.2
```

Aim: write the program to execute operation of the linear algebra

Program:

```
import numpy as np

# Creating matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Addition
add = A + B

# Subtraction
sub = A - B

# Matrix Multiplication
mul = np.dot(A, B)

# Transpose
transpose_A = A.T

# Determinant
det_A = np.linalg.det(A)

# Inverse
inv_A = np.linalg.inv(A)

# Display results
print("Matrix A:\n", A)
print("Matrix B:\n", B)

print("\nAddition:\n", add)
print("\nSubtraction:\n", sub)
print("\nMultiplication:\n", mul)
print("\nTranspose of A:\n", transpose_A)
print("\nDeterminant of A:", det_A)
print("\nInverse of A:\n", inv_A)
```

Output:

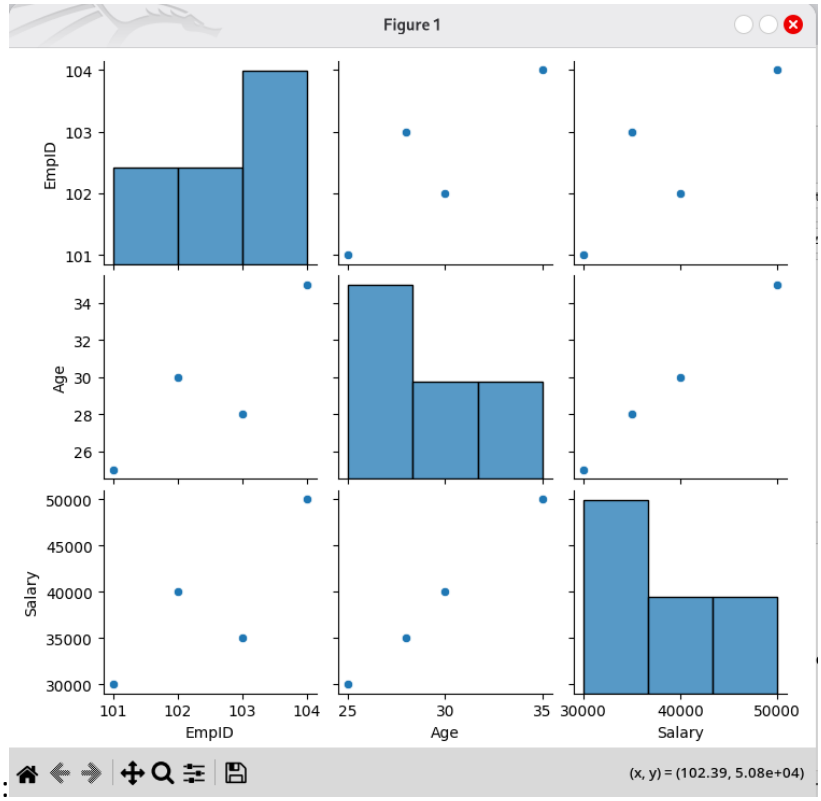
```
┌──(.venv)─(savvy㉿syntaxsavvy)-[~/Program/python]
└─$ python Numpy.py
Matrix A:
 [[1 2]
 [3 4]]
Matrix B:
 [[5 6]
 [7 8]]

Addition:
 [[ 6  8]
 [10 12]]

Subtraction:
 [[-4 -4]
 [-4 -4]]

Multiplication:
 [[19 22]
 [43 50]]

Transpose of A:
 [[1 3]
 [2 4]]

Determinant of A: -2.0000000000000004

Inverse of A:
 [[-2.   1. ]
 [ 1.5 -0.5]]
```

Aim: write a program to plot pair graph to manage employee data using sets

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# Employee data using sets (each tuple = one employee)
employee_data = {
(101, 25, 30000),
(102, 30, 40000),
(103, 28, 35000),
(104, 35, 50000)
}
# Convert set to list for DataFrame
data_list = list(employee_data)
# Create DataFrame
df = pd.DataFrame(data_list, columns=["EmpID", "Age", "Salary"])
# Display data
print(df)
# Pair plot
sns.pairplot(df)
# Show graph
plt.show()
```

Output:

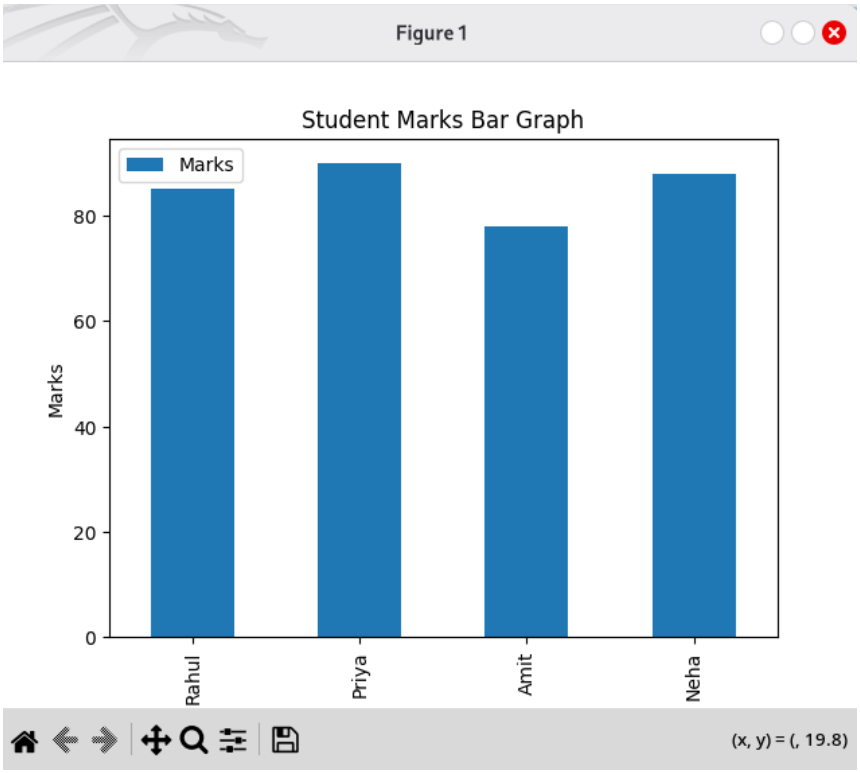


Aim: program to plot bar grapgh using dataframe in python

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

# Creating DataFrame
data = {
'Name': ['Rahul', 'Priya', 'Amit', 'Neha'],
'Marks': [85, 90, 78, 88]
}
df = pd.DataFrame(data)
# Plot bar graph
df.plot(kind='bar', x='Name', y='Marks')
# Labels and title
plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks Bar Graph")
# Show graph
plt.show()
```

Output:

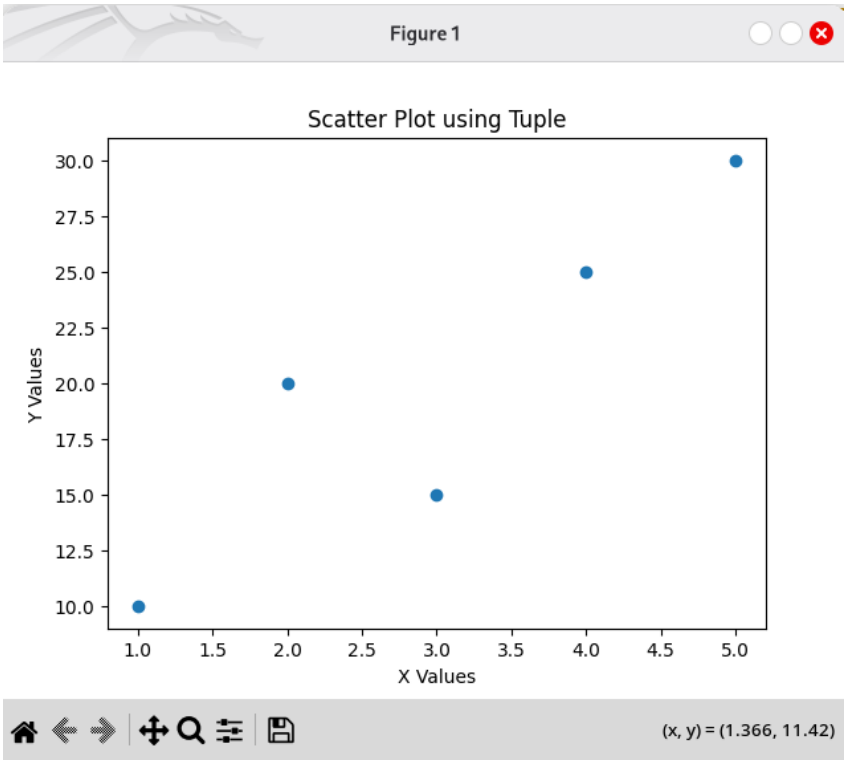


Aim: write a program to plot scatter graph using tuple

Program:

```
import matplotlib.pyplot as plt

# Data stored in tuples
x = (1, 2, 3, 4, 5)
y = (10, 20, 15, 25, 30)

# Plot scatter graph
plt.scatter(x, y)

# Labels and title
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Scatter Plot using Tuple")

# Show graph
plt.show()
```

Output:



Aim: write a program to demonstrate how to catch and except handle in python

Program:

```
try:
# Taking input
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

# Performing division
result = num1 / num2

print("Result:", result)

# Handling division by zero
except ZeroDivisionError:
print("Error: Cannot divide by zero")

# Handling invalid input
except ValueError:
print("Error: Invalid input! Please enter numbers only")

# Executed if no exception occurs
else:
print("Division successful")

# Always executed
finally:
print("Program execution completed")
```

Output: